

WINTERSEMESTER 2025/26

Studiengang/Abschluss: Wirtschaftsinformatik / B.Sc.
Prüfungsfach: Programmierung / Programmierung 2
Prüfungsnummer: **5706 / 55743**
Prüfer: Prof. Dr. Andreas Biesdorf
Anzahl der Arbeitsblätter: 10 Seiten (ohne Deckblätter)
Bearbeitungszeit: 90 Minuten

Mit Bleistift oder in roter Farbe geschriebene Ausführungen werden nicht gewertet!

Zugelassene Hilfsmittel:
Das im Rahmen der Vorlesung erstelle Hilfsmittel
Password ZIP-Datei: WI2026§

Matrikelnummer:

--	--	--	--	--	--

Datum der Klausur:

09.02.2026 **PC-N°:** _____

Punkte: _____ von 60 möglichen

Note: _____

Handzeichen des Prüfers: _____

Hinweise zu Klausuren im FB Wirtschaft

- Mit Aufnahme der Prüfung bringen Sie zum Ausdruck, dass Sie sich für prüfungstauglich halten. Ein Rücktritt von der Klausur während der Bearbeitungszeit ist damit nur noch in ganz besonderen Ausnahmefällen möglich.
- Sollten Sie zu Beginn der Bearbeitungszeit prüfungstauglich sein, dann aber während der Bearbeitungszeit prüfungsuntauglich werden, müssen Sie die Klausur abbrechen, Ihre Prüfungsuntauglichkeit den Aufsichtführenden anzeigen und schnellstmöglich, in jedem Fall aber fristgerecht für ein ärztliches Attest sorgen.
- Mit der Abgabe Ihrer Klausur bringen Sie zum Ausdruck, dass Sie sich während der gesamten Bearbeitungszeit für prüfungstauglich gehalten haben. Ein Rücktritt ist dann nicht mehr möglich.
- Jacken und Taschen sind in ausreichender Entfernung vom Platz zu deponieren.
- Die Bearbeitungszeit beginnt erst, wenn alle Klausuren ausgeteilt sind.
- Es sind nur die zugelassenen Hilfsmittel zu verwenden.
- Das Verlassen des Raumes für Toilettengänge während der Klausur ist nur nach Meldung und Bestätigung der Aufsicht gestattet. Die vorübergehende Abwesenheit wird im Protokoll vermerkt.
- Es sind keine anderen als die ausgegebenen Bearbeitungsblätter zu benutzen.
- Auf dem Tisch sind nur Schreibutensilien erlaubt, keine Mäppchen.
- Jeder Teilnehmer muss sich mit einem Studierendenausweis oder Lichtbildausweis ausweisen.
- Geräusche und Störungen jeder Art sind zu vermeiden.
- Die Klausurensätze sind grundsätzlich nicht zu öffnen. Wer den Klausurensatz öffnet, trägt dafür Sorge, dass die Blätter (innerhalb der Bearbeitungszeit) sortiert und neu geheftet werden. Das Risiko des Verlustes von einzelnen Blättern trägt der Studierende!
- Es ist nicht erlaubt, Aufgabenstellungen auf gesondertes Papier abzuschreiben.
- Mobiltelefone, Smartphones und Smartwatches sind nicht erlaubt!
- Alle Studierenden bleiben auf ihren Plätzen und verhalten sich ruhig, bis die Klausuren vollständig eingesammelt sind. Folgen Sie den Anweisungen der Aufsichtsführenden.
- Bei Täuschung oder versuchter Täuschung wird mit „nicht ausreichend“ bewertet.

Aufgabe 1

- a) **4+1 Sichten nach Kruchten:** Benennen Sie die Sichten nach Kruchten und ordnen Sie die folgenden Architekturaspekte der korrekten Sicht zu: (a) Klassen und Objekte, (b) Akteure und Anwendungsfälle, (c) Performanz und Skalierbarkeit, (d) Hardware-Topologie, (e) Dateien und Repositories. **3 Punkte**

Klassen und Objekte

Akteure und Anwendungsfälle

Performanz und Skalierbarkeit

Hardware-Topologie

Dateien und Repositories

- b) Nach der Einführung der Software folgt die Betriebsphase, die in allen Phasenmodellen in der Regel den längsten Zeitraum im Softwarelebenszyklus einnimmt. [a] Welchen prozentualen Anteil nimmt die Betriebsphase an den Gesamtkosten eines Softwareprojekts über die gesamte Lebensdauer ein? [b] Erläutern Sie die Höhe des Werts im Verhältnis zu den übrigen Phasen. [3 Punkte]

[a] Prozentualer Anteil an den Gesamtkosten eines Softwareprojekts:

[b] Erläuterung:

Aufgabe 2

1) Gegeben sei folgende Spezifikation und Implementierung:

prog_ws25/src/aufgabe02a.py

```
def normalize_and_pack(values, limit):  
    """  
    Alle negativen Werte werden verworfen. Alle verbleibenden Werte werden auf ganze  
    Zahlen gerundet (Standardrundung von Python). Falls nach dem Verwerfen nichts  
    übrig bleibt, ist das Ergebnis die leere Liste. Anschließend werden die Werte in  
    der ursprünglichen Reihenfolge zu „Paketen“ zusammengefasst, sodass die Summe  
    eines Pakets jeweils kleiner oder gleich limit ist. Sobald der nächste Wert  
    nicht mehr in das aktuelle Paket passt, beginnt ein neues Paket. Ist limit <= 0,  
    soll eine Exception geworfen werden.  
  
    :param values: ist eine Liste von Zahlen (int oder float) und darf leer sein  
    :param limit: ist eine ganze Zahl  
    """  
    assert type(values) == list  
    if limit < 0:  
        raise ValueError("limit must be positive")  
  
    cleaned = []  
    for v in values:  
        if v >= 0:  
            cleaned.append(round(v))  
  
    packs = [[]]  
    for x in cleaned:  
        if sum(packs[-1]) + x < limit:  
            packs[-1].append(x)  
        else:  
            packs.append([x])  
  
    if packs == [[]]:  
        return []  
    return packs
```

- a) **Black Box Testentwurf:** Leiten Sie aus der Spezifikation mindestens sechs sinnvolle, nicht redundante Testfälle ab. Verwenden Sie dabei natürliche Partitionen und Randbedingungen, und geben Sie zu jedem Testfall konkrete Eingaben und das erwartete Ergebnis an. Mindestens zwei Testfälle müssen Extremwerte oder Grenzwerte abdecken. **(3 Punkte)**

- b) **Äquivalenzklassen und Grenzwerte:** Begründen Sie Ihre Testauswahl aus a), indem Sie für `values` und `limit` geeignete Äquivalenzklassen benennen und zu mindestens drei Klassen jeweils einen Grenzwertest formulieren. (3 Punkte)

- c) **White Box Analyse:** Prüfen Sie den Code auf Abweichungen von der Spezifikation. Nennen Sie mindestens drei konkrete Fehler oder Spezifikationsverletzungen und erklären Sie jeweils kurz, wie sie sich in falschem Verhalten äußern. Ein Teil Ihrer Analyse muss sich auf Pfadüberdeckung beziehungsweise Zweigüberdeckung beziehen. (3 Punkte)

- d) **White Box Testfälle:** Konstruieren Sie eine *minimale Menge* an Testfällen, die für die gegebene Implementierung alle relevanten Verzweigungen abdeckt. Berücksichtigen Sie dabei insbesondere die Empfehlungen zum Testen von Verzweigungen und Schleifen, also den Fall, dass eine Schleife gar nicht, genau einmal und mehrfach durchlaufen wird. Geben Sie für jeden Testfall kurz an, welche Codepfade oder Zweige er abdeckt. (3 Punkte)

- e) **pytest und Exceptions:** Ergänzen Sie in der Datei `prog_ws25/test/test_aufgabe02.py` die korrespondierenden Tests mit aussagekräftigen Testnamen mithilfe von `pytest`. Prüfen Sie bei Fehlerfällen explizit die Exception. Ergänzen Sie in der Funktion `normalize_and_pack` zusätzlich mindestens zwei Assertions, die als defensive Programmierung am Anfang der Funktion sinnvoll wären, und begründen Sie kurz, welche Fehlerklasse dadurch früh erkennbar wird. (3 Punkte)

Aufgabe 3

Gegeben ist die Beispieldatei `log.txt`, die Zugriffe auf einen Webservice protokolliert. Jede Zeile hat das Format:

```
YYYY-MM-DD HH:MM:SS;LEVEL;USER;ACTION
```

Beispielzeilen:

```
2025-05-10 08:12:03;INFO;alice;login
2025-05-10 08:12:10;ERROR;bob;upload
2025-05-10 08:13:01;WARNING;alice;download
```

Schreiben Sie ein kleines Analyse Tool für die Kommandozeile mit folgenden Anforderungen:

Das Programm akzeptiert folgende Kommandozeilenargumente (Pflichtargumente):

- `--input` oder `-i`: Pfad zu einer Textdatei im Log-Format
- `--output` oder `-o`: Name der Ausgabedatei, in die das Ergebnis geschrieben wird
- `--befehl` oder `-b`: Gibt den auszuführenden Befehl an

Zulässige Befehle:

- *actions*: zählt, wie viele Logzeilen pro ACTION vorkommen
- *user*: zählt, wie viele Logzeilen pro USER vorkommen
- *level*: zählt, wie viele Logzeilen pro LEVEL vorkommen

Die Wahl der Bibliothek (`getopt` / `argparse`) ist Ihnen freigestellt.

Beispielaufruf:

```
python3 aufgabe03.py -i log.txt -o ergebnis.txt -b level
```

Erwarteter Inhalt in `ergebnis.txt`:

```
Befehl: level
ERROR: 3
WARNING: 5
INFO: 12
```

Gegeben ist die Beispieldatei `log.txt`, welche eine Log-Datei einer Software darstellt.

Schreiben Sie ein kleines Analyse-Tool für die Kommandozeile (**10 Punkte**)

Aufgabe 4

Wir möchten Wörter effizient nach ihren Präfixen verwalten, zum Beispiel für Autovervollständigung. Dazu verwenden wir einen Präfixbaum [Trie]. Jeder Knoten speichert seine Kindknoten in einem Dictionary `kinder` und ein Flag `ist_wortende`, das markiert, ob an dieser Stelle ein vollständiges Wort endet.

Regeln:

1. Ein Wort wird *Zeichen für Zeichen* in den Trie eingefügt.
2. Fehlt ein Kindknoten für ein Zeichen, wird er angelegt.
3. Ein Wort gilt nur dann als enthalten, wenn der Pfad existiert und am Endknoten `ist_wortende == True` ist.
4. Die Methode `woerter_mit_praefix(praefix)` soll alle Wörter in alphabetischer Reihenfolge liefern, die mit `praefix` beginnen.

Gegeben ist der folgende unvollständige Code in `prog_ws25/src/aufgabe04.py`:

prog_ws25/src/aufgabe04.py

```
class Node:
    def __init__(self):
        self.kinder = {} # dict[str, Node]
        self.ist_wortende = False

class Trie:
    def __init__(self):
        self.wurzel = Node()

    def einfuegen(self, wort):
        """
        Fuegt ein Wort in den Trie ein.
        """
        aktueller = self.wurzel
        for ch in wort:
            if ch not in aktueller.kinder:
                aktueller.kinder[ch] = Node()
            aktueller = aktueller.kinder[ch]
        aktueller.ist_wortende = True

    def enthaelt(self, wort):
        """
        True, wenn wort vollstaendig enthalten ist (ist_wortende am Ende).
        """
        aktueller = self.wurzel
        for ch in wort:
            if ch not in aktueller.kinder:
                return False
            aktueller = aktueller.kinder[ch]
        return aktueller.ist_wortende

    def woerter_mit_praefix(self, praefix):
        """
        Liefert alle Woerter, die mit praefix beginnen, alphabetisch sortiert.
        """
        aktueller = self.wurzel
        for ch in praefix:
            if ch not in aktueller.kinder:
```

```
        return []
        aktueller = aktueller.kinder[ch]

    ergebnis = []
    self._sammle_woerter(aktueller, praefix, ergebnis)
    return ergebnis

def _sammle_woerter(self, knoten, gebaut, ergebnis):
    """
    Hilfsfunktion: sammelt ab knoten alle Woerter ein.
    gebaut ist das bisher aufgebaute Wort.
    """
    pass

trie = Trie()
woerter = ["cat", "car", "cart", "dog", "dot", "dove"]
for w in woerter:
    trie.einfuegen(w)

print(trie.woerter_mit_praefix("ca"))
print(trie.woerter_mit_praefix("do"))
print(trie.enthaelt("car"), trie.enthaelt("care"))
```

Beispieleingaben /Ausgaben:

```
woerter_mit_praefix("ca") → ["car", "cart", "cat"]
woerter_mit_praefix("do") → ["dog", "dot", "dove"]

enthaelt("car") → True
enthaelt("care") → False
```

- a) Ergänzen Sie die Methode `_sammle_woerter(self, knoten, gebaut, ergebnis)`, sodass alle Wörter ab `knoten` gesammelt werden. Achten Sie darauf, dass nur dann ein Wort in `ergebnis` aufgenommen wird, wenn `knoten.ist_wortende == True` ist. Die Traversierung der Kinder muss alphabetisch erfolgen. **(10 Punkte)**
- b) Ergänzen Sie die Methode `woerter_mit_praefix(self, praefix)` so, dass sie bei leerem Präfix alle Wörter im Trie zurückgibt und dabei ebenfalls alphabetisch sortiert ist. **(7 Punkte)**

Hinweis: Nutzen Sie `sorted(knoten.kinder.keys())` für die Iteration in alphabetische Reihenfolge.

Aufgabe 5

Die Elemente einer Liste sollen nach zwei Kriterien sortiert werden. Primär aufsteigend nach der Länge des Strings, sekundär bei gleicher Länge alphabetisch aufsteigend.

Gegeben ist folgender, leider unvollständiger, Code:

prog_ws25/src/aufgabe05.py

```
def mergesort(input_liste):
    """
    Sortiert die Strings in input_liste
    1) aufsteigend nach ihrer Laenge
    2) bei gleicher Laenge alphabetisch aufsteigend
    und gibt die sortierte Liste zurueck.
    """
    pass

def merge(left, right):
    """
    Fuehrt zwei bereits sortierte Listen zusammen und gibt Liste sortiert zurueck
    Sortierkriterium:
    - zuerst len(s)
    - bei Gleichstand: alphabetisch
    """
    result = []
    i = 0
    j = 0

    while i < len(left) and j < len(right):
        pass

    result.extend(left[i:])
    result.extend(right[j:])
    return result

# Testen Sie Ihre Implementierung mit dem folgenden Code
word_list = ["Hallo", "Python", "Wirtschaftsinformatiker", "Aufgabe", "Sortieren",
             "Kurs", "AI", "Zoo", "Auto"]

print(mergesort(word_list))
```

- Implementieren Sie `mergesort(input_liste)` rekursiv, inklusive sinnvoller Abbruchbedingung und korrekter Aufteilung in zwei Teillisten. Die Funktion muss die sortierte Liste zurückgeben und die vorgegebene `merge` Funktion nutzen. **[7 Punkte]**
- Ergänzen Sie in `merge(left, right)` die fehlende Vergleichslogik, sodass nach folgendem Kriterium sortiert wird: zuerst aufsteigend nach `len(s)`, bei gleicher Länge alphabetisch aufsteigend. **[5 Punkte]**

Viel Erfolg!